

Campus resources, connected.

University Lab & Equipment Booking System

A web application that connects resource discovery, university booking rules, staff approval, equipment handover, return tracking, and usage analytics.

The problem

Shared laboratories and equipment are difficult to coordinate through paper forms and messages. This system gives students, faculty, and staff a common record of availability, ownership, approval, and return condition.

The complete journey

Discover resources > choose a department, date and time > check capacity, rules and availability > submit a request > staff approval or automatic reservation > issue resources > record the return > update analytics and history.

Who uses it

Students and faculty: discover resources, request bookings, track outcomes, cancel eligible requests, and receive notifications.

Lab staff: review department requests, manage resource status, issue equipment, and record returns.

Coordinators: manage department resources, booking rules, priorities, and analytics.

Administrators: manage users, permissions, departments, categories, university resources, rules, and activity history.

Implementation

React 19 and React Router power the Vite frontend. Express handles the API. SQLite is accessed through Node.js node:sqlite and repository modules. JWT, bcrypt, Zod, Helmet, rate limits, and server-side role checks protect the workflow. A scheduled job creates in-app reminders.

A clear separation of responsibilities.

Location	Purpose
client/src/pages	Public home, authentication, dashboard, resource discovery, booking, operations, analytics, notifications, and administration screens.
client/src/components	Shared branding, page headings, status badges, cards, loading states, and keyboard-accessible dialogs.
client/src/context + api	Authentication state and the HTTP client that sends authenticated API requests.
client/src/styles/global.css	Responsive design, Manrope headings, DM Sans body type, and the plum / white / lime visual palette.
server/src/routes + controllers	HTTP endpoints, permission checks, validation entry points, and responses.
server/src/services	Booking, conflict detection, rules, recommendations, equipment handover, notifications, and analytics.
server/src/models + db	SQLite repository wrappers and shared database access.
server/src/config	Database schema, migrations, and environment configuration.
server/src/jobs	Booking reminders, return reminders, and overdue status updates.
server/tests/workflow.cjs	Isolated in-memory regression tests; the saved application database is untouched.

Designed to keep every step accountable.

Booking and intelligent recommendations

bookingService checks the requested department, participant capacity, rules, and resource availability. availabilityService checks overlapping reservations and equipment quantities. recommendationService scores available labs by department, capacity, facilities/purpose, and equipment sufficiency. Conflicts return later available time windows where possible.

Issue, return, and maintenance

issueService creates a dated handover record, checks physical stock, and records the return time and condition. Late returns can trigger an account restriction. Damaged or missing equipment is conservatively blocked for inspection until staff restore its maintenance state. Activity logs preserve decisions and resource changes.

Verification completed

16 in-memory integration checks cover capacity, department restrictions, duplicate requests, concurrent approval and automatic reservation conflicts, recommendations, alternatives, issue state, overdue cancellation, damaged returns, notifications, analytics, cancelled-request approval, and invalid time windows. Browser smoke checks cover the administration pages and key mobile pages; these checks are not a complete security audit.

Run locally

Install Node.js 22.5 or newer and project dependencies. Configure server/.env and client/.env using the examples. Run npm run dev. The frontend defaults to port 5173 and the API to port 5000. SQLite initializes automatically. The one-time setup screen creates the first administrator. Optional npm run seed resets the database and must only be used for disposable demo data.

Deployment and submission

Run one API process with persistent SQLite storage and a strong JWT secret. Configure HTTPS, client origin, backups, frontend history fallback, and VITE_API_URL for the deployed API. Multiple API workers require database-level locking; the current reservation queue protects one process. The separate demonstration video and GitHub or deployed application link still need to be prepared for submission.